Neural Networks 206 (2027) 109797

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Parallelizable memory recurrent units

Florent De Geeter *, Gaspard Lambrechts , Damien Ernst , Guillaume Drion

Montefiore Institute, University of Liege, Liege, 4000, Belgium

ARTICLE INFO

Keywords:
Memory
Multistability
Parallelization
Recurrent neural network
Surrogate gradient

ABSTRACT

With the emergence of massively parallel processing units, parallelization has become a desirable property for new sequence models. The ability to parallelize the processing of sequences with respect to the sequence length during training is one of the main factors behind the uprising of the *Transformer* architecture. However, Transformers lack efficiency at sequence generation, as they need to reprocess all past timesteps at every generation step. Recently, *state-space models* (SSMs) emerged as a more efficient alternative. These new kinds of recurrent neural networks (RNNs) keep the efficient update of the RNNs while gaining parallelization by getting rid of nonlinear dynamics (or recurrence). SSMs can reach state-of-the-art performance through the efficient training of potentially very large networks, but still suffer from limited representation capabilities. In particular, SSMs cannot exhibit persistent memory, or the capacity of retaining information for an infinite duration, because of their monostability. In this paper, we introduce a new family of RNNs, the *memory recurrent units* (MRUs), that combine the persistent memory capabilities of nonlinear RNNs with the parallelizable computations of SSMs. These units leverage multistability as a source of persistent memory, while getting rid of transient dynamics for efficient computations. We then derive a specific implementation as proof-of-concept: the bistable memory recurrent unit (BMRU). This new RNN is compatible with the parallel scan algorithm. We show that BMRU achieves good results in tasks with long-term dependencies, and can be combined with SSMs to create parallelizable hybrid networks that have both transient dynamics and persistent memory.

1. Introduction

Recurrent neural networks (RNNs) (Cho et al., 2014; Hochreiter & Schmidhuber, 1997) used to be the most popular architecture for tackling sequential tasks thanks to their ability to model a large range of dynamical systems, until being foreshadowed by the Transformer architecture (Vaswani et al., 2017). The impossibility to parallelize RNNs with respect to time for training made Transformers a much suited architecture for building large models and training them on large amount of data. However, for generative modeling tasks, the Transformer suffers from a high computational complexity. Indeed, at every timestep of generation, it needs to process the full sequence of past inputs. RNNs have however regained interest in recent years thanks to the success of state-space models (SSMs) (Gu et al., 2022). SSMs use strictly linear dynamics in cascade with static nonlinearities, making their computation easily parallelizable. This allows SSMs to be used in large networks and to be trained on large datasets, because they can benefit from the computational power offered by GPUs. SSMs can compete with Transformers on a variety of tasks (Gu et al., 2022), and large-language models based on SSMs like Mamba (Gu & Dao, 2024) have achieved comparable or better performance than Transformers for similar model sizes. SSMs are more efficient than Transformers during inference, as their recurrent formulation allows them to compute each output only from their previous output and the input, while Transformers require the whole sequence of inputs to be fed (or cached activations along the whole sequence of inputs). The strictly linear recurrent dynamics of SSMs however creates some limitations: SSMs can only approximate fading-memory systems, i.e. systems that have a unique stable state (Boyd & Chua, 1985). Monostable systems such as SSMs encode past information in their transient dynamics, which creates a memory that ineluctably fades over time and makes some tasks unsolvable for a fixed number of layers (Merrill et al., 2024). Alternatively, multistable systems have several stable states, which allows to encode information for an infinite duration. It is possible to foster multistability in RNNs (Lambrechts et al., 2023; Vecoven et al., 2021), and multistability has been shown to drastically improve RNN performance especially in long-term dependencies tasks (Lambrechts et al., 2023). Multistability however requires strongly nonlinear recurrent connections, which usually prevents parallelization. In this configuration, multistable RNNs cannot compete with parallelizable alternatives such as SSMs or Transformers.

* Corresponding author at: University of Liege, Montefiore Institute, Belgium.
E-mail addresses: florent.degeeter@uliege.be (F. De Geeter), gaspard.lambrechts@uliege.be (G. Lambrechts), dernst@uliege.be (D. Ernst), gdrion@uliege.be (G. Drion).

https://doi.org/10.1016/j.neunet.2026.109797
Received 20 January 2026; Received in revised form 24 June 2026; Accepted 22 October 2026
Available online 23 October 2026
0893-6080/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

This work attempts to close this gap through the creation of novel RNN models that are parallelizable over the sequence length but exhibit persistent memory through multistability. Section 2 first introduces the concept of RNNs with internal clocks, i.e. RNNs in which internal states can be updated several times between two consecutive timesteps, as a way to formalize our design approach. Section 3 focuses on the RNN class where infinite updates can occur between two consecutive timesteps. From that, it introduces the concept of *memory recurrent unit* (MRU), a class of RNNs that exhibit persistent memory but no fading memory. Section 4 further introduces *bistable memory recurrent unit* (BMRU), a concrete example of MRU that can be trained on sequential tasks. BMRU properties and performance are finally evaluated on several benchmarks in Section 5.

2. RNNs with internal clocks

RNNs encode time-dependencies in a recurrent hidden state h_t whose update depends on the previous hidden state h_{t-1} and current observation of a time-series x_t. It writes

$$h_t = f_\theta(x_t, h_{t-1}; \theta), \forall t \geq 1 \tag{1}$$

where f_θ represents the update equation of the RNN and θ the parameters of the network. The key mechanism is the recurrent connection that defines a dependency between h_t and h_{t-1}. Nonlinear RNNs such as LSTM (Hochreiter & Schmidhuber, 1997) and GRU (Cho et al., 2014) have remained state-of-the-art methods in sequence modeling for several decades.

The classical RNN formulation of Eq. (1) constraints the internal state h_t to only be updated when the network receives a new input x_t, i.e. only once every timestep t. This constraint forbids the network to modify its internal state evolution between two timesteps. It particularly impedes our goal to quickly encode information on stable equilibria at convergence.

To release this constraint, we define a class of RNNs whose internal dynamics are decoupled from the external dynamics of the input time-series. Each RNN of this class has internal clocks that permit to update their internal states N-times between two timesteps t, where N can be different for each unit. The internal update equation of such RNN unit between two timesteps writes

$$\tilde{h}_t[0] = h_{t-1}, \tag{2a}$$
$$\tilde{h}_t[n] = f_\theta(x_t, \tilde{h}_t[n-1]; \theta), \forall n \geq 1, \tag{2b}$$
$$h_t = \tilde{h}_t[N], \tag{2c}$$

where t is the current timestep, $n \in [0, \dots, N]$ the current internal clock iteration and $N \in \mathbb{N}$ the number of internal clock iterations. $\tilde{h}_t[n]$ is the transient internal state value at iteration n and h_t the final internal state at time t.

Classical RNNs are contained within this class. Indeed if we set $N = 1$, Eq. (2b) becomes

$$\tilde{h}_t[1] = f_\theta(x_t, \tilde{h}_t[0]; \theta)$$

with $\tilde{h}_t[0] = h_{t-1}$ and $\tilde{h}_t[1] = h_t$, which leads to Eq. (1).

For $1 < N < \infty$, we have the set of RNNs whose internal dynamics are decoupled from the time-series dynamics, as their internal state is updated N times between two timesteps. Such RNNs can be trained as classical RNNs, but the potential increase in expressive power comes at a computational cost, as the effective sequence length becomes $N \cdot T$ at inference, T being the length of the time-series. Using linear recurrence of the form

$$f_\theta(x_t, \tilde{h}_t[n]; \theta) = A(\theta)\tilde{h}_t[n-1] + B(\theta)x_t,$$

can mitigate this complexity, as the value $\tilde{h}[N]$ can be computed analytically from $\tilde{h}[0]$ while maintaining the possibility to output potentially complex trajectories. This is for instance the approach taken by practical implementations of state-space models (SSMs) (Smith et al., 2022).

For $N \to \infty$, we have the specific set of "convergent" RNNs, i.e. RNNs that can converge toward their steady-state values between two timesteps. In this work, we use the framework of convergent RNNs to build a novel type of RNN unit that uses stable states as the basis for their computations.

3. The specific case of convergent RNNs ($N \to \infty$)

We focus on RNNs that converge toward stable equilibria, i.e. RNNs whose update equation $f_\theta(x_t, h_{t-1}; \theta)$ is such that:

$$\forall x_t, \tilde{h}_t[0],\ \exists h^* \in \mathbb{R} : \lim_{N\to\infty} f_\theta^N(x_t, \tilde{h}_t[0]; \theta) = h^*,$$

where:

$$f_\theta^n(x_t, \tilde{h}_t[0]; \theta) = \begin{cases} f_\theta(x_t, f_\theta^{n-1}(x, \tilde{h}_t[0]; \theta); \theta) & \text{if } n > 1, \\ f_\theta(x_t, \tilde{h}_t[0]; \theta) & \text{if } n = 1. \end{cases}$$

Letting the internal clock to loop an infinite number of times ($N \to \infty$) ensures that the RNN has converged toward a stable equilibrium between two time steps. This property, which is specific to this class of RNNs, ensures a stationary output, i.e. that the output remains stable over time if the input barely changes. This property is of great interest, as it ensures that the information encoded in the RNN unit will not fade over time. It however requires specific characteristics to be exploitable in practice.

To illustrate this, we consider the following RNN update equation:

$$h_t = (1-c)\cdot h_{t-1} + c \cdot \tanh\big(x_t + (\beta+1)\cdot h_{t-1}\big), \forall t \geq 1, \tag{3}$$

which is a simplified version of the BRC and nBRC derived by Vecoven et al. (2021). To construct the convergent version, we rewrite Eq. (3) at timestep t with an internal clock

$$\tilde{h}_t[0] = h_{t-1} \tag{4a}$$
$$\tilde{h}_t[n] = (1-c)\cdot \tilde{h}_t[n-1] + c \cdot \tanh\big(x_t + (\beta+1)\cdot \tilde{h}_t[n-1]\big), \forall n \geq 1 \tag{4b}$$
$$h_t = \tilde{h}_t[N] \tag{4c}$$

and simulate its response over several internal clock iterations N for different values of previous state value h_{t-1} and two different values of the parameter β (Fig. 1) (other parameter values are $c = 0.01$ and $x_t = 0$). In both simulations, the trajectories corresponding to different values of h_{t-1} are transiently distinct, until they all converge toward a steady-state. The distinct transient trajectories create fading memory, which can encode quantitative information about h_{t-1} but only for a strictly finite amount of time.

Here, we focus our interest on what happens at convergence. In the case of $\beta = -1.5$ (Fig. 1, left), all initial h_{t-1} lead to the same equilibrium point, and the RNN unit cannot maintain any stationary information about h_{t-1}. It is restricted to fading memory. This is because Eq. (3) for $\beta = -1.5$ leads to a monostable system, i.e. a system that has only one stable equilibrium for all input values x_t. Monostable RNNs are incapable of encoding stationary (i.e. persistent) information about the past. It includes all RNNs with linear recurrence, such as SSMs, as well as all gated RNNs in which gates do not depend on past state values and whose update gate z_t is strictly larger than 0 (for the formulation $(1 - z_t) \odot h_{t-1}$) or smaller than 1 (for the formulation $z_t \odot h_{t-1}$) (Feng et al., 2024; Martin & Cundy, 2018; Qin et al., 2023).

In the case of $\beta = 1.5$ (Fig. 1, right), the trajectories converge toward two different equilibria depending on h_{t-1}: $h_t \simeq +1$ for positive values of h_{t-1} and $h_t \simeq -1$ for negative values of h_{t-1}. The RNN unit maintains a stationary, qualitative information about h_{t-1}. In order words, some information about h_{t-1} is encoded in *persistent memory*. This is because Eq. (3) for $\beta = 1.5$ leads to a bistable system, i.e. a system that has two stable equilibria for a set of input values x_t.

Constructing a multistable RNN with infinite internal clock cycles would therefore permit to isolate its persistent memory capabilities. But constructing such RNN by looping a large number of times in Eq. (2b) would drastically slow down inference and breach the convergence

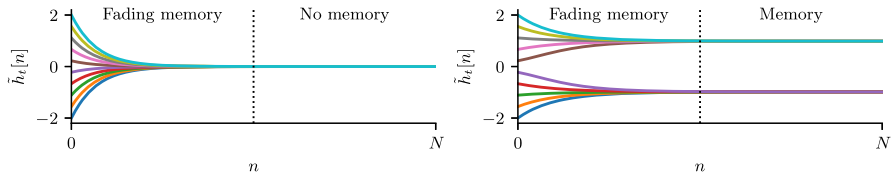


Fig. 1. Monostability vs bistability in a RNN with internal clock. The figure shows internal state trajectories of the RNN unit described by Eq. (3) for different initial conditions $\tilde{h}_t[0] = h_{t-1}$. (**left**) Evolution of the system when $\beta = -1.5$. (**right**) Evolution of the system when $\beta = 1.5$.

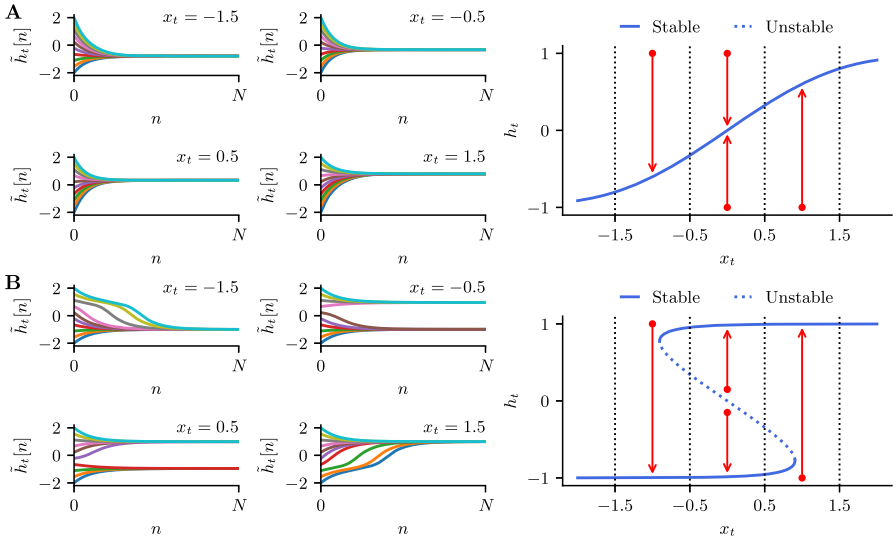


Fig. 2. Convergence properties of the RNN unit described by Eq. (4b) for different values of input x_t and either $\beta = -1.5$ (A) or $\beta = 1.5$ (B) (**left**) Internal state trajectories corresponding to different initial conditions $\tilde{h}_t[0] = h_{t-1}$ and 4 different input values x_t. (**right**) Solutions of the steady-state equation Eq. (5) for $\beta = -1.5$ (**A**) and $\beta = 1.5$ (**B**). The red arrows show convergence trajectories from different initial conditions $\tilde{h}_t[0]$.

guarantee. We rather exploit the fact that the internal state converges toward an equilibria at $N \to \infty$, i.e. $\tilde{h}_t[n] = \tilde{h}_t[n-1] = h_t$. To do so, we rewrite an equivalent formulation for Eq. (4b) at the convergence point

$$\begin{aligned}\tilde{h}_t[0] &= h_{t-1}\\ h_t &= (1-c)\cdot h_t + c\cdot \tanh\big(x_t + (\beta+1)\cdot h_t\big),\end{aligned}$$

which can be rewritten

$$F(h_t, x_t; \beta) = 0; \quad \tilde{h}_t[0] = h_{t-1} \tag{5}$$

where $F(h_t, x_t; \beta) = h_t - \tanh\big(x_t + (\beta+1)\cdot h_t\big)$. Eq. (5) provides a set of necessary conditions that the RNN unit must satisfy at convergence. This equation defines an implicit function that can have multiple solutions h_t for a given input x_t^*. The solution that corresponds to the output at time t depends on the value of h_{t-1}, which dictates toward which stable point h_t^* the cell will converge (i.e., h_{t-1} determines in which basin of attraction the trajectory lies). This solution must also satisfy the stability property to ensure that it is a stable point:

$$\left.\frac{df(x_t, h_t; \beta)}{dh_t}\right|_{x_t^*, h_t^*} < 1$$

where $f(x_t, h_t; \beta) = (1-c)\cdot h_t + c\cdot \tanh\big(x_t + (\beta+1)\cdot h_t\big)$.

The solutions of Eq. (5) and their stability are shown in Fig. 2 for $\beta = -1.5$ (A) and $\beta = 1.5$ (B). For $\beta = -1.5$, the function has only one solution h_t for all x_t, and the convergent RNN has a similar input-output function as a static layer. For $\beta = 1.5$, the function has a memory region where two solutions are possible, and the selected solution h_t depends on h_{t-1}. The RNN has encoded some information of the past input in persistent memory.

More generally, the set of necessary conditions that any RNN defined by Eq. (2b) must satisfy at convergence are determined by the implicit function

$$F_\theta(x_t, h_t; \theta) = 0, \tag{6}$$

where $F_\theta(x_t, h_t; \theta) = h_t - f_\theta(x_t, h_t; \theta)$. It is therefore possible to efficiently compute the outputs of a convergent RNN at each timestep t by using this implicit function as the update function

$$F_\theta(x_t, h_t; \theta) = 0, \forall t = 1, \dots, T, \tag{7}$$

where h_{t-1} is used to select the output value when multiple solutions exist. This leads to a recurrent unit that only encodes information in persistent memory. We call this type of recurrent unit a **memory recurrent unit (MRU)**. It is important to note that Eq. (7) can be solved in parallel for each timestep, as the function only depends on x_t and h_t, but requires h_{t-1} to select the proper solution based on history.

In dynamical systems theory, the implicit function $F_\theta(x_t, h_t; \theta) = 0$ corresponds to the bifurcation diagram of the system described by Eq. (2b). For instance, Eq. (5) and associated Fig. 2 correspond to a hysteresis bifurcation. Hysteresis bifurcations underlie bistability in existing RNN cells such as the bistable recurrent cell (BRC) (Vecoven et al., 2021) (see Appendix A). More generally, a MRU can be built from any nonlinear dynamics by using its bifurcation diagram as an implicit activation function. The output state in the memory region can be computed using iterative methods such as the Newton-Raphson method (Danieli et al., 2025; Gonzalez et al., 2024; Lim et al., 2023). Likewise, implicit differentiation can be used to compute exact gradients during backpropagation. The use of iterative methods can however slow down inference and hinder parallelization. In the next section, we show how this drawback can be circumvented, focusing on the hysteresis bifurcation and its associated bistability.

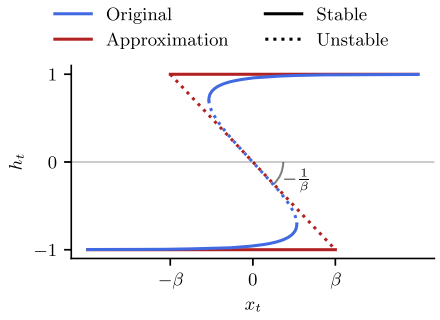


Fig. 3. Comparison between the implicit function and its approximation. This figure compares the solutions (h_t, x_t) of the implicit function defined by Eq. (5) (in blue) with the approximation defined by Eq. (8) (in red) where $\alpha = 1$. Solid lines correspond to stable points, and dashed lines to unstable points.

4. A bistable memory recurrent unit

4.1. A computationally efficient hysteresis-based MRU

Instead of directly using the implicit function derived by the hysteresis bifurcation diagram, we propose to create a MRU based on an approximation of this function that maintains all its qualitative properties, but which is computationally efficient and remain parallelizable. Indeed, unlike the general concept of MRU, this approximation does not require any iterative method as it is formulated as an explicit function that depends on the current input and the previous state. The goal of this section is to present this approximation, and to show how we build a bistable memory recurrent unit (BMRU) around it.

We focus our design on the multistable behavior of the implicit function, i.e. $\beta > 0$, as $\beta \leq 0$ leads to a monostable, memory-less function (see Fig. 2). Under this condition, the original function can be decomposed into three parts: the upper stable points, the lower stable points, and the unstable points that serve as a boundary in the bistable region. We approximate these three parts by modeling the stable points as constant values $\pm\alpha$, and the *boundary* function as a linear function whose slope is the slope of the original function at $(0, 0)$, i.e. $-\frac{1}{\beta}$. The approximation writes

$$h_t = \begin{cases} \alpha \cdot S(x_t) & \text{if } |x_t| \geq \beta, \\ \alpha \cdot S(h_{t-1} + \frac{x_t}{\beta}) & \text{if } |x_t| < \beta, \end{cases} \tag{8}$$

where S denotes the *sign* function (with $S(0) = 1$). Similarly to the original function, h_t is independent of h_{t-1} for a large input, but not for a small input. A comparison between this approximation and the original function is shown in Fig. 3.

The convergence condition ensures that $h_{t-1} = \pm\alpha$ for all timesteps, which permits to further simplify Eq. (8) for $|x_t| < \beta$:

$$h_t = \begin{cases} \alpha \cdot S(x_t) & \text{if } |x_t| \geq \beta, \\ h_{t-1} & \text{if } |x_t| < \beta. \end{cases} \tag{9}$$

It is only every time $|x_t|$ is greater than β that h_t is updated. Otherwise, it will remain constant. This property is illustrated in Fig. 4.

4.2. A learnable bistable memory recurrent unit

Eq. (9) defines the input-output properties of the BMRU, but does not contain any learnable parameters. Here, we add such parameters by taking inspiration from gated RNN structure.

First, we observe that the function defined by Eq. (9) outputs two different values depending on the comparison between the values of $|x_t|$ and β. We can implement this function by introducing a *binary* gate, z_t, that computes this condition, and rewrite Eq. (9) as a gate-dependent update rule. It writes

$$z_t = H(|x_t| - \beta), \tag{10a}$$

$$h_t = z_t \cdot S(x_t) \cdot \alpha + (1 - z_t) \cdot h_{t-1}, \tag{10b}$$

where t is the current timestep, H the Heaviside function and S the sign function. Eq. (10a) computes the binary gate value z_t, which is used to distinguish whether we are inside the bistable region ($z_t = 0$) or outside of it ($z_t = 1$). Eq. (10b) then computes the new state value h_t depending on the value of z_t.

We can then extend this formulation to the multidimensional case, where multiple inputs x_t converge at a layer composed of multiple BMRUs. To be coherent with the variables used in gated RNNs, we name the input values x_t and their combination at each BMRU cell $\hat{h}_t$, the *candidate*. x_t becomes a M-dimensional vector where M is the number of inputs to the layer. Moreover, β can be made input-dependent, and is therefore renamed β_t. $\hat{h}_t$, β_t and h_t become N-dimensional vectors, where N is the number of BMRU cells in the layer. We use a classical fully connected layer to compute the vectors $\hat{h}_t$ and β_t from the input vector x_t, adding the positivity constraint for β_t. The equations of the multidimensional BMRU network write

$$\hat{h}_t = W_x x_t + b_x, \tag{11a}$$

$$\beta_t = |W_\beta x_t + b_\beta|, \tag{11b}$$

$$z_t = H(|\hat{h}_t| - \beta_t), \tag{11c}$$

$$h_t = z_t \odot S(\hat{h}_t) \odot \alpha + (1 - z_t) \odot h_{t-1}, \tag{11d}$$

where $\odot$ is the hadamard product, t is the current timestep, W_x and W_β are matrices of learnable parameters, b_x, b_β and α are learnable parameters, H is the Heaviside function and S is the sign function.

The use of S and H makes the use of backpropagation difficult, as their gradient is 0 everywhere except in 0 where it is ∞. However, there exist solutions to overcome this problem, and we chose the *surrogate gradient* approach: the non-differentiable functions are used in the forward pass, but the derivatives of other functions, which are differentiable and similar to the non-differentiable ones, are used in the backward pass. This technique is notably used in the context of *spiking neural networks* (Eshraghian et al., 2023; Neftci et al., 2019).

For the Heaviside function, we chose the following surrogate gradient (inspired from Eshraghian et al., 2023)

$$\frac{df}{dx}(x) = \frac{1}{1 + (\alpha_{surr}\pi x)^2}, \tag{12}$$

where α_{surr} is a tunable parameter. We note that this surrogate derivative comes from the function

$$f(x) = \frac{1}{\pi\alpha_{surr}} \operatorname{atan}(\alpha_{surr}\pi x) + \frac{1}{2}. \tag{13}$$

Fig. 5 plots $f(x)$ and $\frac{df}{dx}(x)$ for different values of α_{surr}. The derivative exhibits a localized peak centered at x = 0. As the parameter α_{surr} increases, the width of this peak decreases, making it more concentrated. The case $\alpha_{surr} = 0$ is special and leads to a constant derivative equal to 1, which is also called the *straight-through estimator* (Bengio et al., 2013). The surrogate gradient used for S is simply $2 \cdot \frac{df}{dx}(x)$, as S has the same shape as H with outputs being either -1 or 1. Appendix D shows how α_{surr} can impact the learning of BMRU networks.

4.3. Properties of the bistable MRU

BMRU is a special recurrent cell compared to the usual ones and it has some interesting properties that are highlighted here.

Stationarity. BMRU is by design stationary, meaning that repeating the same input several times will not impact state value. It makes state update independent of input duration when the input is constant. One advantage that comes from this stationarity property is the ability to generalize to longer inputs. Fig. 6 illustrates this on a simple benchmark, where the goal is to retain a binary value ± 1 given at $t = 0$ for some time, during which the input is 0. This benchmark can be easily solved using either persistent memory (Fig. 6, left) or fading memory (Fig. 6,

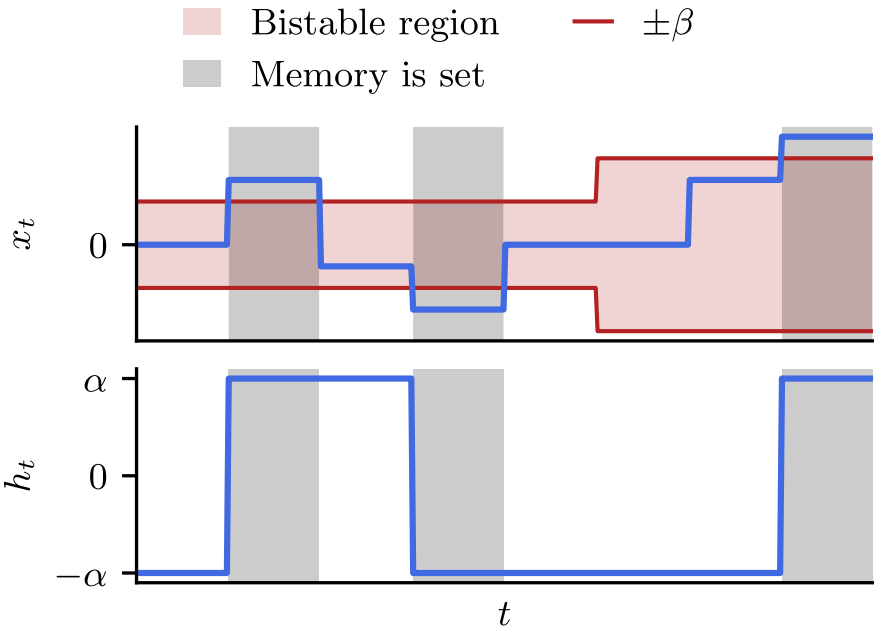


Fig. 4. Example simulation of the cell defined by Eq. (9) for different values of x_t and β. The top graph defines two illustrative variations of x_t and β, as well as the bistable region defined by β. The bottom graph shows the effect of variations in x_t and β on the evolution of the state h_t. The gray areas highlight the timesteps at which the memory is updated, which occurs when x_t is outside of the bistable region.

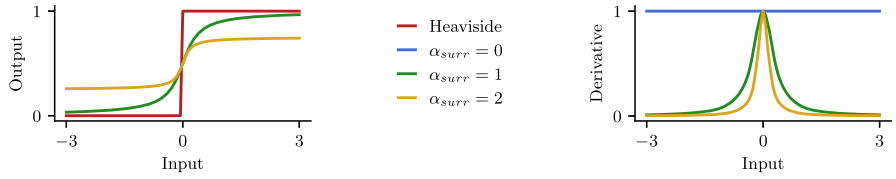


Fig. 5. Surrogate gradient used in BMRU. (left) Comparison between the Heaviside function and the function defined by Eq. (13) used to approximate the gradient for different values of α_{surr}. **(right)** Impact of α_{surr} on the surrogate gradient defined by Eq. (12).

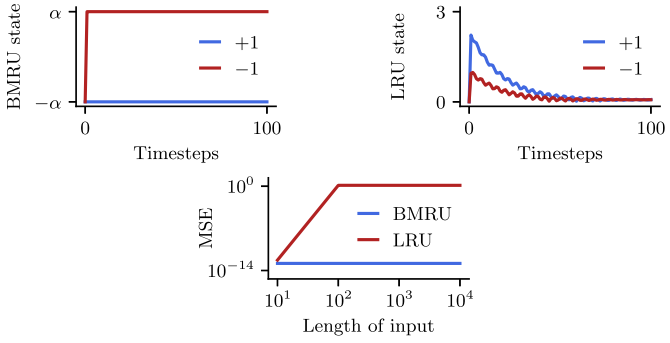


Fig. 6. Persistent memory of BMRU compared to the fading memory of LRU. Small models with one LRU or BMRU layer of one unit have been trained on a simple benchmark whose inputs start with ±1 followed by 0's. The goal of the models is to output the first input at the last timestep. **(left, center)** Evolution of the states of LRU and BMRU with respect to the timesteps for the two possible inputs. As the LRU state is complex, its norm is plotted. **(right)** MSE computed with both models on the two possible inputs for different sequence lengths.

center). However, BMRU encoding the information in stable states, it remains encoded forever, and the performance is not impacted by the input length (Fig. 6, right). LRU however encodes the information in fading memory, which does not generalize to larger input length, as memory fades over time.

No vanishing nor exploding gradient in the memory region. At every timestep, the state h_t will be either set equal to h_{t-1} or to a value independent of h_{t-1}. When the previous state is kept, the derivative of the new state with respect to the previous one is 1. When the memory is updated, the derivative is 0, and therefore the gradient will not go further back in time during the backward pass. In other words, the information is either kept intact, or overwritten. Mathematically, it writes

$$\frac{\partial h_t}{\partial h_{t-1}} = \begin{cases} 1 & \text{if } z_t = 0, \\ 0 & \text{if } z_t = 1. \end{cases}$$

Fig. 7 shows how the gradient of the last state h_T with respect to the first candidate $\hat{h}_1$ or the first β_1 evolves with respect to the sequence length. As expected, it stays constant, with two possible values: a null one if the memory has been updated at a later timestep, or a non-zero value if the memory was never updated after the first timestep. Indeed, in this case, each derivative $\frac{\partial h_t}{\partial h_{t-1}}$ is equal to 1, therefore their product is equal to 1, ensuring a constant gradient across time.

Compatibility with parallel scan. BMRU update equations can be rewritten using an associative operator, therefore allowing the use of the parallel scan, as proved in Appendix B. A *scan* is an operation that takes a binary operator $\circledast$ and an ordered set of n elements $[a_0, \dots, a_{n-1}]$ and returns the ordered set

$$\left[a_0, (a_0 \circledast a_1), (a_0 \circledast a_1 \circledast a_2), \dots, (a_0 \circledast a_1 \circledast \dots \circledast a_{n-1})\right].$$

The goal of the parallel scan is to perform a scan with a better time complexity than by performing it sequentially, decreasing the complexity of computing the n outputs from $O(n)$ to $O(\log(n))$ on a GPU.

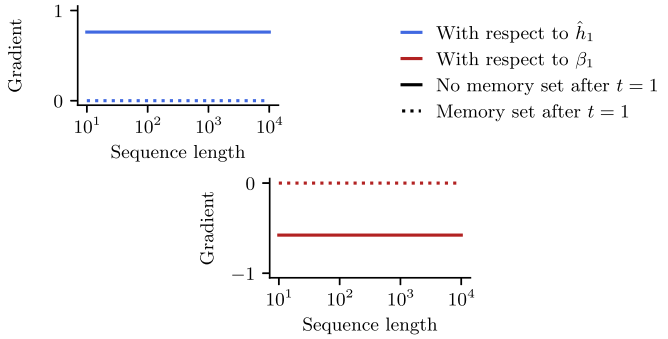


Fig. 7. Consistency of the gradient of BMRU with respect to time. Evolution of the gradient of the last state h_T with respect to the first candidate $\hat{h}_1$ or β_1. Two cases are considered: either only the first timestep sets the memory, i.e. $|\hat{h}_1| > \beta_1$ and $|\hat{h}_t| < \beta_t \; \forall t > 1$, either another timestep also sets the memory.

5. Experiments

This section aims at analyzing the performance of BMRU on several regression and classification benchmarks, with increasing level of difficulty. The idea is to showcase the properties, strengths but also the limitations of BMRU. Networks of LRU (Orvieto et al., 2023) were also trained to highlight differences between the SSMs and BMRU. Each value presented in this section is computed by taking the average over 5 runs.

5.1. Model architecture

The architecture follows the one used in the SSM literature (Gu et al., 2022; Orvieto et al., 2023). It consists of a linear layer that projects the input to some dimensions (called the *model dimensions*, H), followed by recurrent blocks and fully-connected layers applied timestep-wise to extract the predictions. Each recurrent block has a batch normalization layer, recurrent cell (either BMRU or LRU), a GLU activation and a skip connection. These recurrent cells have their own number of dimensions (*state dimensions*, N), which can differ from the model dimensions. More specifically, they receive inputs in H dimensions, update their N dimensional state and generate an output with H dimensions. The recurrent cells can also receive positional embeddings that encode the current timestep into some vector of arbitrary dimensions. Unless it is specified otherwise, we consider the prediction of the model to be its output at last timestep, as our main goal is to evaluate the ability of the models to retain information. The parameters of each experiment are given in Appendix C.

5.2. Copy-first-input

The first benchmark is purely synthetic and consists of remembering a real value for some duration while being perturbed by noise. In practice, models receive a two-dimensional input $x_t = (r_t, f_t)$ at each timestep. The first dimension is a real value r_t independently and identically distributed from N(0,1) at all timesteps. The second one is a flag f_t indicating if r_t has to be retained ($f_t = 1$) or not ($f_t = 0$). This flag is only set at the first timestep. Models are trained by computing the MSE between the first input and their last output. In this experiment, we used networks of 2 recurrent blocks, with $H = N = 256$ and without any positional encoding. All the models are trained on 60000 samples, 10% of which are used for validation. The test set also consists of 60000 samples.

Fig. 8 shows the test MSE obtained by the networks on two versions of the benchmark, one where information has to be retained for 100 timesteps, the other for 300 timesteps. For the small duration, both models learn. However, for the longer duration, only BMRU manages to learn the task. Although increasing network depth would possibly make

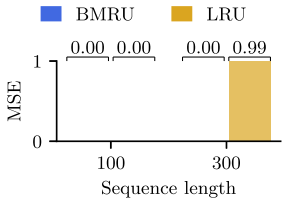


Fig. 8. Results on the copy-first input benchmark. Test MSEs obtained by BMRU and LRU models on the copy-first-input benchmark for two sequence lengths: 100 and 300.

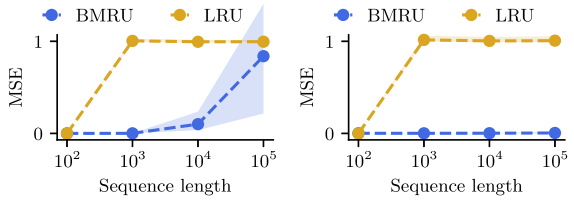


Fig. 9. Generalization capabilities of BMRU and LRU with respect to the sequence length. MSEs obtained by the BMRU and LRU models trained on the copy-first-input with a sequence length of 100 when they are evaluated on longer sequences with two different levels of noise. (**left**) The noise is sampled from $\mathcal{N}(0, 1)$. (**right**) The noise is sampled from $\mathcal{N}(0, 0.1)$.

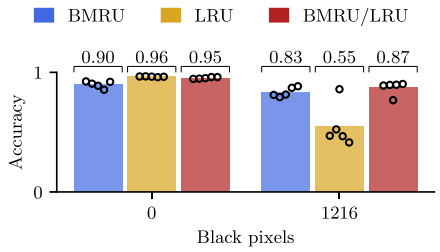
LRU learn, our goal here is not to optimize the hyperparameters to get the best loss possible, but rather to highlight the ability of BMRU to retain information for a long duration even with a shallow network.

This benchmark also allows to highlight generalization property of BMRU to longer sequences. One can evaluate this generalization by taking the models trained on 100 timesteps and testing them on longer sequences. To do that, we created small test sets whose sequences lengths are a power of 10, starting from 10^2 to 10^5. Each test set is composed of 6000 samples. Fig. 9 shows the evolution of the MSEs obtained by both cells on these small test sets with respect to the sequences length. LRU does not generalize to larger sequences well, as it encodes the information in fading memory. BMRU is much more resilient as it encodes the information in persistent memory, and the performance only decreases if the stored values are overwritten by high amplitude noise (center graph). If the noise is sufficiently low, the performance is unaffected by the sequence length (right graph).

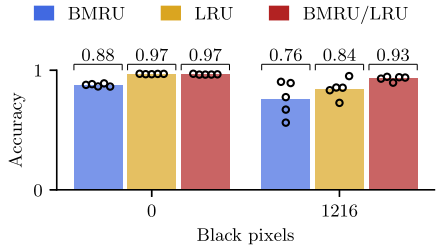
5.3. Permuted sequential MNIST

The MNIST dataset is one of the most well-known datasets (Lecun et al., 1998). In this experiment, we use a variant of MNIST, called the Sequential MNIST (Le et al., 2015), where the pixels are fed to the models one by one. While being known as an *easy* benchmark, the Sequential MNIST is still relevant when testing new architectures. To increase the difficulty, the pixels are shuffled before being given to the models. Compared to the previous benchmark, models must combine information received at different timesteps in order to predict the correct label. To also assess the memorization capabilities of the models, we add, in some experiments, black pixels at the end of the sequences. This forces the models to not only combine temporal information but also to retain it for a long period. In practice, 1216 black pixels are added, bringing the sequence lengths to 2000. Furthermore, we add positional encodings to the inputs of BMRU, as we observed that it can greatly improve its performance. On the other hand, we observed that adding these encodings to LRU actually impedes its performance, therefore these are only given to BMRU in these experiments.

Fig. 10 shows the accuracies obtained by BMRU (blue bars) and LRU models (yellow bars), with or without black pixels, and for two network depths. All recurrent blocks contain 256 neurons, as in the previous experiment. The performance of LRU is better than BMRU on

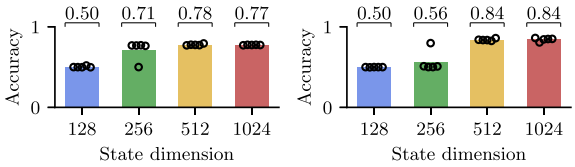


(a) 2 rec. blocks

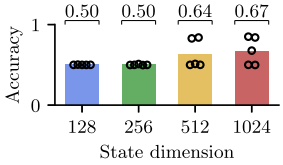


(b) 3 rec. blocks

Fig. 10. Results on the permuted sequential MNIST. Accuracies obtained by BMRU, LRU and BMRU/LRU models on the permuted sequential MNIST, with or without black pixels added at the end of the sequences. Models with 2 and 3 recurrent block have been tested.



(a) 1 rec. block (b) 2 rec. blocks



(c) 3 rec. blocks

Fig. 11. Results on the pathfinder benchmark. Each plot shows the accuracies obtained by BMRU models on the pathfinder benchmark, for different state dimensions (x-axes) and network depths (1, 2 and 3 recurrent blocks from left to right, respectively).

the version without black pixels, but BMRU better maintains its performance when black pixels are added, especially for the more shallow model. LRU indeed requires more depth to handle longer dependencies, whereas BMRU only needs two layers to handle these long dependencies. On the other hand, this experiment also shows that LRU is much better at combining the information from the different timesteps. This motivates our next experiment: as LRU is better at combining the information using fading memory, and BMRU at retaining it using persistent memory, their combination should get the best of both worlds, while still being parallelizable. The results obtained using this combination are shown in Fig. 10 (red bars). Note that for a fair comparison, the state dimension of each cell has been divided by 2, in order to maintain the number of neurons in each recurrent block at 256. We can see that the combination has the same performance than LRU alone when there are no black pixels, but its performance does not decrease when black pixels are added. It even stays higher than BMRU alone. This highlights the potential of combining the fading memory and persistent memory properties of these two types of parallelizable RNNs. For more studies on this hybrid layer, Appendix E evaluates the impact of using different ratios of BMRU/LRU neurons. Moreover, Appendix F studies how making the parameter α input-dependent influences performance.

5.4. Pathfinder

The pathfinder benchmark is part of the *long-range arena* group of benchmarks (Tay et al., 2020). It consists of 32x32 black and white images where lines are drawn randomly, as well as two dots. The goal of the benchmark is to predict if the two dots are connected by a line or not. Images are fed pixel by pixel, which leads to sequences of 1024 timesteps. The goal of this experiment is to test the capabilities of shallow BMRU networks, i.e. max 3 recurrent blocks, to solve such a difficult benchmark without trying to achieve state-of-the-art performance through parameter tuning. We chose a similar model approach as in the S4 and LRU papers (Gu et al., 2022; Orvieto et al., 2023): the predictions of the models are the means of their outputs (instead of their last timestep), and the recurrent layers are bidirectional.

Fig. 11 shows the accuracies obtained by BMRU on this benchmark. Note that as there are only two classes, so an accuracy of 5081 corresponds to random guess. First, the results show that the BMRU is able to learn on the Pathfinder benchmark, even with only one recurrent layer. Second, these experiments show that the depth of the network is not that important, but its width, i.e. the state dimension, has a bigger impact on performance. This differs from SSMs, for which it is known that their ability to handle longer time dependencies grows with the network depth. The drop of performance when more layers are added can be explained by the *discretization* implied by BMRU and the simplicity of layer interconnections. Indeed, as each neuron of BMRU can only output two values ($\pm\alpha$), each BMRU layer adds more discretization, and makes the learning more difficult.

6. Discussion

The concept of MRU and BMRU are at the junction of several topics that are discussed in this section and compared to relevant works.

Multistable RNNs. Adding multistability in RNNs is not a very explored topic from the point of view of machine learning. Some works have been done to better understand multistability in RNNs from a dynamical system point of view (Cheng et al., 2006; Krishnamurthy et al., 2022), even some that includes *thresholding* functions (Edwards, 2000; Tournoy & Doiron, 2022). However, these works do not include any machine learning experiments. Few works have highlighted how this property can improve the memorization capability of RNNs, either by building new types of RNNs (Vecoven et al., 2021) or by enforcing multistability in existing RNNs (Lambrechts et al., 2023). This work attempts to highlight the interesting properties of multistable RNNs for sequence learning and its complementarity with the more classical fading memory.

Parallelizable RNNs. Parallelization is a *sine qua non* condition nowadays, and is one the features of Transformers that put them in the front of the scene. This motivates also the research in SSMs. Since the creation of SSMs (Gu et al., 2022, 2021), numerous papers have added their contributions and improvements, which finally led to Mamba (Gu & Dao, 2024), the first SSM architecture that was used in large language models. Some approaches augment SSMs to improve their capacity to handle temporal dependencies. For instance, Sun et al. (2025) adds a second smaller SSM inside the usual one. This new SSM generates a gate at every timestep, which is then used to combine the n previous states into one vector. This approach reminds of the attention mechanism, and while it helps RNNs at remembering past information, it does not solve the fading memory problem. However, it provides a nice way to combine information from nearby timesteps and might complement BMRU

to alleviate its weaknesses. Other works tried to make RNN parallelizable, but most of them end up removing the nonlinearities (Beck et al., 2024; Feng et al., 2024; Martin & Cundy, 2018). There exist works that developed parallelizable RNNs, but not without concession: They had for example to remove the time-dependency in the RNN update (Jiang et al., 2021) or to limit the depth of the network (Zini et al., 2021). Recent and promising approaches reformulate the RNNs equations as a system and use iterative methods to solve it (Danieli et al., 2025; Gonzalez et al., 2024; Lim et al., 2023). In this work, we show that multistable RNNs can be parallelizable over the sequence length if we consider the case of convergent RNNs, i.e. RNNs that reach full convergence between two timesteps.

Surrogate gradients. The usage of non-differentiable functions in neural networks does not happen often, therefore surrogate gradients are not that useful in classic deep learning. However, in some fields with restrictions, it allows to benefit from all the advantages of back-propagation while using special functions. For instance, in the topic of *spiking neural networks*, where outputs must be0*′*s and1*′*s, surrogate gradients have allowed the networks to be trained like any classical networks (Eshraghian et al., 2023; Lee et al., 2016; Neftci et al., 2019; Nieves & Goodman, 2021). However, the impact of using an approximation during backward pass is difficult to measure, and therefore makes the usage of surrogate gradients delicate. Also referred to as *pseudo-gradient*, this idea is not really new: Bengio et al. (2013) introduces the *straight-through estimator* uses a constant pseudo-gradient, i.e. the derivative of a linear activation, while Zeng et al. (1993) and Goodman and Zeng (1994) use the derivative of a sigmoid to train MLPs and RNNs with threshold units.

Steady-states and equilibria. While BMRU does not really rely on the computation of steady-states, the notion of MRU is built around it and the concept of implicit function. This reminds of *Deep Equilibrium Models* (DEQs) introduced by Bai et al. (2019) where layers are formulated as an implicit function and outputs are the steady-states of this implicit function. They use a solver to estimate this steady-state starting from some initial guess. However, this initial guess does not depend on any past information, and the implicit function are typically monostable, therefore these layers do not implement any memory. A MRU can be seen as a variant of DEQs where the implicit function is multistable and the initial guess is chosen to be the previous steady-state.

Binary gates. BMRU emulates the bistability property through the binary gate z. Other works also rely on binary gates to improve the memorization of RNNs. For instance, Yin et al. (2026) generates a sequence of binary gates with a rhythmic module, allowing to not update all the components of the hidden state at every timestep. An other example is the work of Shan et al. (2023) that introduces a binary gate and a specific training algorithm to learn optimal recurrent routes.

Improving MRUs and BMRU. While BMRU already achieved appreciable performance on several benchmarks, some concerns may be raised about its capacity to address more complex tasks. Indeed, we already mention its difficulty to scale to deeper networks, as well as the computational limitation imposed by the quantization and finiteness of the state space of BMRU. However, more recent works based on MRUs and BMRU have increased the learning capabilities of these approaches by creating new variants of BMRU or by improving the model architecture. These new versions allows deeper networks of MRUs to be trained and achieve excellent performance on several benchmarks including language modeling tasks. Notably, Fyon et al. (2026) shows that a 6-layers BMRU network performs well on a language modeling task, and also that BMRU can be adapted to run on specific hardware without compromising performance. Furthermore, Brandoit et al. (2026) introduces a variant of BMRU, the cumulative MRU (CMRU), that relies on an infinite number of stable states rather than only two and achieve comparable performance to SSMs. This work also shows that the quantization of states does not impede performance. Finally, Bakija et al. (2026) extends the idea of generalization briefly mentioned in Section 4.3 in the context of reinforcement learning and shows how BMRU allows agents to generalize to much longer environments without any retraining. It also highlights the advantage of the hybrid model in tasks that both require integrating information distributed at different timesteps and remembering information over long periods. This body of work demonstrates the great potential of MRUs in various contexts, including complex ML benchmarks, embedded AI or reinforcement learning.

Future works. In the experiments section, we highlighted three characteristics of BMRU: its property to extend its memory to much longer durations, the gain of performance that can be obtained when combining it with a SSM, and its ability to learn long-term dependencies in difficult benchmarks with shallow networks. All of these deserve to be explored. For instance, we know that SSMs are better in deep networks, which is not the case of BMRU, therefore making mixed models where recurrent blocks are made of more SSM layers than BMRU layers could lead to interesting results. Also, this degradation of performance when more BMRU layers are added could be explored, as improvements in the cell equations, the initialization or the training algorithm (Zhang et al., 2026) could be made to improve learning in deeper models. As mentioned previously, the works of Brandoit et al. (2026), Fyon et al. (2026) already improve the scaling of MRUs to deeper networks. Furthermore, BMRU equations have three interesting properties: the quantization of the state, the shape of the update decision, which is a comparison between $\hat{h}_t$ and β_t, and finally the possibility for the update gate z_t to be null. The first one is necessary to have discrete stable states, the second one to approximate the hysteresis bifurcation and the last one to have persistent memory (as $z_t = 0$ implies $h_t = h_{t-1}$). However, these can be used independently of the others. The impact of each definitively deserves to be explored, especially the third one which allows for persistent memory, as we are not aware of another RNN architecture that allows for this property. In addition to that, BMRU update rule is restrictive: either we keep the past state, or we totally forget it. However, nothing prevents from combining the past state and the input, as long as the dependency with respect to the past state stays linear. The CMRU mentioned in the previous section is an example of this. Also, we note that for introducing the MRU, we put forward a concept of RNNs with internal clocks, the MRU being a specific implementation of such cells when the internal clock iterates an infinite number of times. In this respect, we believe it may also be potentially interesting to further exploit this concept when the clock iterates a finite number of times. Furthermore, the usage of surrogate gradients is practically inexistent in classic deep learning, as all computations are differentiable. While it seems that using surrogate gradients may impede the performance of neural networks by introducing some mismatches between forward and backward passes, we wonder if using non-differentiable functions in classical neural networks may increase the performance and robustness of these networks. Indeed, step functions like the Heaviside one are more resilient to small changes in their output, but they probably make the training more difficult. Finally, BMRU was only tested on classification and regression tasks, but one of its advantages shared with SSMs is its efficient sequence generation capacity. It would be interesting to try it on generation tasks like text generation benchmarks for instance, or in reinforcement learning tasks as done by Bakija et al. (2026).

7. Conclusion

In this paper, we introduce the concept of *memory recurrent units* (MRU): a new class of RNNs that do not exhibit any transient dynamics but that creates persistent memory through multistability. We also present a concrete implementation of a MRU, the *bistable memory recurrent unit* (BMRU), derived from the hysteresis bifurcation. The equations of BMRU are closed to the usual gated RNN equations, and are compatible with the parallel scan algorithm.

We observe that BMRU can achieve good results in different benchmarks, all requiring learning long-term dependencies. Moreover, combining BMRU with a SSM leads to a parallelizable recurrent model that has both linear transient dynamics and a multistable behavior, which allows to efficiently encode temporal information for very long durations. Indeed, the linear dynamics can encode complex information but this memory will unavoidably fade over time, while multistability encode more qualitative information in a never-fading memory.

Finally, BMRU has shown to be efficient with shallow networks, while SSMs typically requires more layers to learn long-term dependencies.

To conclude, MRUs are a new concept that deserve to be explored and especially since we showed it could lead to a well working implementation, the BMRU, that showed interesting properties and performance. This paves the way for new experiments and improvements for future designs.

CRediT authorship contribution statement

Florent De Geeter: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Conceptualization; **Gaspard Lambrechts:** Writing – review & editing; **Damien Ernst:** Writing – review & editing, Supervision; **Guillaume Drion:** Writing – review & editing, Writing – original draft, Supervision, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been the subject of patent applications (Numbers: EP26151077 and EP26175248.9). Florent De Geeter gratefully acknowledges the financial support of the Walloon Region for Grant No. 2010235 - ARIAC by DW4AI. Gaspard Lambrechts is a postdoctoral researcher of the *Fund for Scientific Research* (FNRS) from the *Wallonia-Brussels Federation* in Belgium. The present research benefited from computational resources made available on Lucia, the Tier-1 supercomputer of the Walloon Region, infrastructure funded by the Walloon Region under the grant agreement n°1910247. This work was supported by the Belgian Government through the Federal Public Service Policy and Support.

Appendix A. Hysteresis bifurcation in the bistable recurrent cell

The bistable recurrent cell (Vecoven et al., 2021) is described the set of equations

$$c_t = \sigma(U_c x_t + w_c \odot h_{t-1} + b_c), \tag{A.1a}$$

$$a_t = 1 + \tanh\big(U_a x_t + w_a \odot h_{t-1} + b_a\big), \tag{A.1b}$$

$$h_t = c_t \odot h_{t-1} + (1 - c_t) \odot \tanh\big(U_h x_t + a_t h_{t-1} + b_h\big), \tag{A.1c}$$

where c_t is the update gate, a_t the feedback gate, and $U_c, w_c, b_c, U_a, w_a, b_a, U_h, b_h$ are learnable parameters. At steady-state, we have $h_t = h_{t-1}$, and Eq. (A.1c) writes

$$(1 - c_t) \odot h_t = (1 - c_t) \odot \tanh\big(U_h x_t + a_t h_t + b_h\big),$$

which, by replacing a_t by Eq. (A.1b) at steady-state and dividing each term by $(1 - c_t)$ (c_t being the output of a sigmoid function, its value is strictly smaller than 1), leads to

$$\tanh\big(U_h x_t + (1 + \tanh\big(U_a x_t + w_a \odot h_t + b_a\big))h_t + b_h\big) - h_t = 0. \tag{A.2}$$

This steady-state function is a static, implicit function that relates the values of the input x_t to the values of the state h_t at convergence. We can show that this function corresponds to a hysteresis singularity with x_t as the bifurcation parameter and b_a as the unfolding parameter. Fig. A.12 illustrates the unfolding of the hysteresis bifurcation for $b_a = -1$ (monostable), $b_a = 0$ (singular) and $b_a = 1$ (bistable) (other parameter values are $U_h = 1$, $U_a = w_a = b_h = 0$). Modifications of the parameters $U_h \neq 0, U_a, w_a, b_h$ lead to translations and deformations of the bifurcation plot without alteration of the existence of a hysteresis singularity.

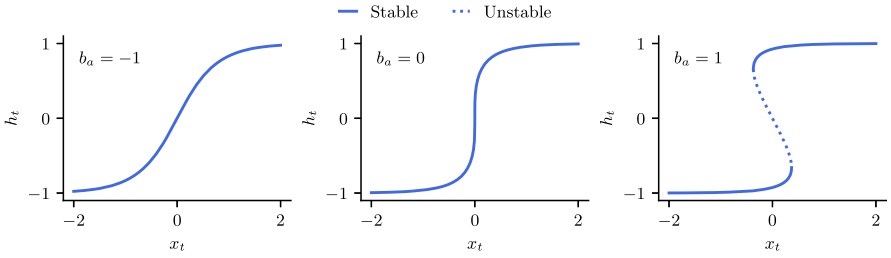


Fig. A.12. Hysteresis bifurcation in the bistable recurrent cell. This figure shows the solutions to the implicit function defined by Eq. (A.2) as well as their stability for three values of b_a.

Appendix B. Compatibility of BMRU with the parallel scan algorithm

The *scan* operation, also called the *prefix sum* operation, computes from a binary operator $\oplus$ and an array of n elements $[a_0, a_1, \dots, a_{n-1}]$ the array

$$[a_0, a_0 \oplus a_1, \dots, a_0 \oplus a_1 \oplus \dots \oplus a_{n-1}].$$

A naive implementation of the scan consists in a simple loop over the input array that accumulates the results and stores them in the output array. This implementation has a time complexity of $O(n)$. However, it is possible to perform the scan with a better time complexity when the operator $\oplus$ is associative and when multiple processors are available.

The parallel scan algorithm introduced by Blelloch (1990) is an algorithm that performs the *scan* operation on a array of n elements with a time complexity of $O(\frac{n}{p} + \log(p))$, where p is the number of processors. Assuming this algorithm is run on a GPU, we can consider $p = n$, which therefore gives a time complexity of $O(\log(n))$. Notably, this algorithm is used with the SSMs during their training (Smith et al., 2022).

This section proves that BMRU equations are compatible with the parallel scan algorithm.

Theorem 1. *The parallel scan can be used to perform the scan on an array with a binary operator $\oplus$ if and only if $\oplus$ is associative.*

Proof. The proof is given in (Blelloch, 1990). □

Furthermore, Blelloch (1990) showed that there exists a binary associative operator that computes the ordered set of states from a first-order linear recurrence.

Theorem 2. *Assume a first-order linear recurrence of the following form:*

$$h_t = \begin{cases} b_0 & \text{if } t = 0, \\ a_t \odot h_{t-1} + b_t & \text{if } 0 < t < T, \end{cases}$$

where a_t and b_t $\forall t \in [0, T]$ are independent of $h_{t'}$ $\forall t' \in [0, T]$. Consider the set of pairs $c_t = [a_t, b_t]$ and the binary operator $\circledast$ defined as follows:

$$c_t \circledast c_{t'} \equiv [c_{t,a} \odot c_{t',a}, c_{t',a} \odot c_{t,b} + c_{t',a}]$$

where $c_{t,a}$ and $c_{t,b}$ are the first and second elements of c_t. Then,

1. *The operator $\circledast$ is associative,*
2. *Performing the scan with the operator $\circledast$ on the array $[c_0, \dots, c_T]$ creates the array $[s_0, \dots, s_T]$ where $s_t = [y_t, h_t]$ and y_t is defined as:*

$$y_t = \begin{cases} a_0 & \text{if } t = 0, \\ a_t \odot y_{t-1} & \text{if } 0 < t < T. \end{cases}$$

It results that the parallel scan can be used to solve this first-order linear recurrence as $\circledast$ is associative (point 1.), and the solutions h_t will be the second values of the generated pairs s_t (point 2.).

Proof. The proof is given in (Blelloch, 1990), Section 4.1. □

To prove that BMRU is compatible with the parallel scan, we can therefore show that Eq. (11a), (11b), (11c), (11d) can be rewritten as a first-order linear recurrence.

Theorem 3. *Assume a sequence of inputs $[x_1, \dots, x_T]$ and fixed parameters $W_x, W_\beta, b_x, b_\beta$ and α. Eq. (11a), 11b, 11c, 11d describe a first-order linear recurrence, and are therefore compatible with the parallel scan.*

Proof. First, let us separate the equations that are independent of the previous state h_{t-1} from the ones that are not. Eq. (11a), (11b), (11c) do not depend on h_{t-1}. Therefore $\hat{h}_t$, β_t and z_t can be computed in parallel for all timesteps.

The last equation, Eq. (11d), is the only one that has to be analyzed. As a reminder, here it is:

$$h_t = z_t \odot S(\hat{h}_t) \odot \alpha + (1 - z_t) \odot h_{t-1}.$$

The left term is independent of h_{t-1}, hence let us define the set of b_t's as follows:

$$b_t \equiv z_t \odot S(\hat{h}_t) \odot \alpha.$$

Also, the factor that multiplies h_{t-1} is independent of it. Let us define the set of a_t's as follows:

$$a_t \equiv 1 - z_t.$$

Eq. (11d) can therefore be rewritten using a_t and b_t:

$$a_t = 1 - z_t, \tag{B.1a}$$

$$b_t = z_t \odot S(\hat{h}_t) \odot \alpha, \tag{B.1b}$$

$$h_t = \begin{cases} h_0 & \text{if } t = 0, \\ a_t \odot h_{t-1} + b_t & \text{if } 0 < t < T. \end{cases} \tag{B.1c}$$

Once again, Eq. (B.1a), (B.1b) are independent of h_{t-1} and can be computed in parallel for all timesteps. Finally, Eq. (B.1c) is a first-order linear recurrence and by using theorem 2, parallel scan can be used to solve it. □

Appendix C. Training parameters

This section gives all the training parameters used in the experiments of Section 5: Tables C.1–C.3 respectively give the parameters of Section 5.2–5.4.

Table C.1
Training parameters used in Section 5.2.

Parameter	Value(s)
Number of samples in dataset	60000
Sequence length	100 - 300
Train / valid ratio	90% / 10%
Epochs	100
Learning rate	Cosine annealing: $10^{-4} \rightarrow 10^{-3}$ during 10 first epochs, then $10^{-3} \rightarrow 10^{-5}$
Weight decay	0.0001 for BMRU parameters 0.05 for other parameters
Number of recurrent blocks	2
Model dim	256
State dim	256
Positional encoding	No
Activation between blocks	GLU
Bidirectional	No
Number of fully connected layers (after last block)	2
Pooling for prediction	Last timestep
(BMRU) α_{surr}	1
(LRU) r_{min}, r_{max} and θ_{max}	0.0, 0.99 and 2π

Table C.2
Training parameters used in Section 5.3.

Parameter	Value(s)
Number of black pixels	0 - 1216
Pixel normalization	$p/255 - 0.5$, with $p \in [0, 255]$
Train / valid ratio	90% / 10%
Epochs	100
Learning rate	Cosine annealing: $10^{-4} \rightarrow 10^{-3}$ during 10 first epochs, then $10^{-3} \rightarrow 10^{-5}$
Weight decay	0.0001 for BMRU parameters 0.05 for other parameters
Number of recurrent blocks	2 - 3
Model dim	256
State dim	256
Positional encoding	16 dim, only for BMRU
Activation between blocks	GLU
Bidirectional	No
Number of fully connected layers (after last block)	2
Pooling for prediction	Last timestep
(BMRU) α_{surr}	1
(LRU) $r_{\min}$, $r_{\max}$ and $\theta_{\max}$	0.0, 0.99 and 2π

Table C.3
Training parameters used in Section 5.4.

Parameter	Value(s)
Pixel normalization	$(p - \mu)/\sigma$, with $p \in [0, 255]$, $\mu = 10.94$ and $\sigma = 38.51$
Train / valid ratio	90% / 10%
Epochs	100
Learning rate	Cosine annealing: $10^{-4} \rightarrow 10^{-3}$ during 10 first epochs, then $10^{-3} \rightarrow 10^{-5}$
Weight decay	0.0001 for BMRU parameters 0.05 for other parameters
Number of recurrent blocks	1 - 2 - 3
Model dim	256
State dim	128 - 256 - 512 - 1024
Positional encoding	16 dim
Activation between blocks	GLU
Bidirectional	Yes
Number of fully connected layers (after last block)	2
Pooling for prediction	Mean of all timesteps
(BMRU) α_{surr}	1

Appendix D. Impact of α_{surr} on the performance

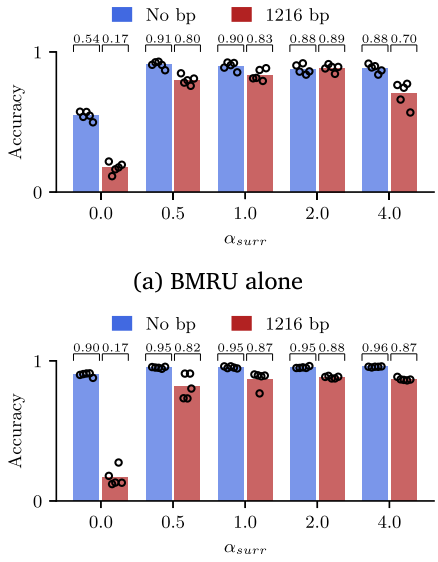


Fig. D.13. Impact of α_{surr} on the performance. Accuracies obtained by BMRU and BMRU/LRU models on the permuted sequential MNIST, with or without black pixels added at the end of the sequences, and for different values of α_{surr}. All models have two recurrent layers with a size of 256.

In Section 4.2 we introduced the concept of *surrogate gradient* as a solution to the impossibility to train BMRU because of its use of functions with ill-defined derivatives, i.e. the Heaviside and the sign functions. The surrogate gradient used in the experiments is defined by Eq. (12) and consist of a peak centered at $x = 0$ with a maximal value of 1. The tunable parameter α_{surr} dictates the width of this peak, as shown in Fig. 5. The case $\alpha_{surr} = 0$ is a special case as it leads to a constant derivative of 1. We used $\alpha_{surr} = 1$ in the experiments of Section 5. However, it may have an impact on the performance reached by the models. This appendix therefore evaluate this impact by training different models (with only BMRU or hybrid) on the permuted sequential MNIST (with and without black pixels) for different values of α_{surr}.

Fig. D.13 shows the mean accuracies obtained by these models and emphases how adding black pixels can degrade more or less degrade the performance depending on α_{surr}. A null value leads to poor performance, which suggests the importance of weighting the gradients depending on how close the inputs are from the step. The other values all lead to models that learn well, but the extreme values, i.e. 0.5 and 4.0 tends to leads to worse accuracies. The two last values, 1.0 and 2.0, give similar results.

Appendix E. Impact of the BMRU/LRU ratio on the performance

In Section 5.3 we combined BMRU with LRU to create hybrid recurrent layers, with the goal of bringing together their strengths and mitigating their weaknesses. The combination indeed helped improving the performance of models. However, we only tested hybrid layer including the same proportion of BMRU and LRU neurons. This appendix evaluates on the same benchmark, i.e. the permuted sequential MNIST with and without black pixels, how changing this ratio impacts the performance.

Fig. E.14 shows the mean accuracies obtained for different ratios. While having too many neurons from one type tends to slightly degrade the performance, the intermediate values leads to similar results. This suggests that the performance of hybrid models is not very sensitive to the ratio between the two types of layers, at least for this task.

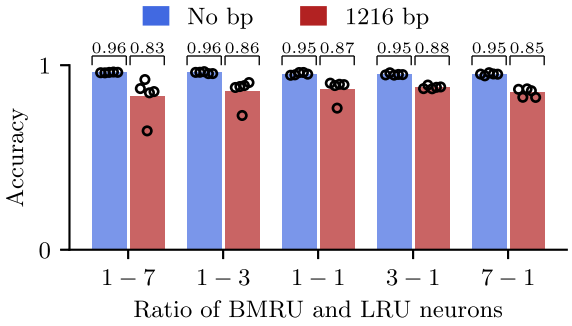
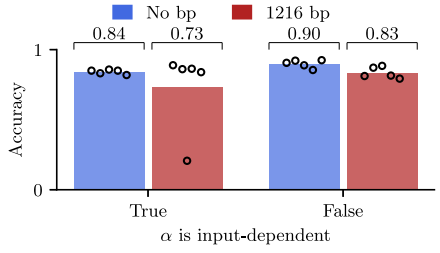
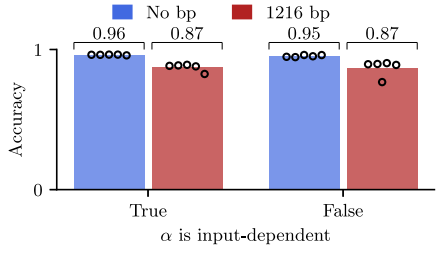


Fig. E.14. Impact of the BMRU/LRU ratio on the performance. Accuracies obtained by hybrid models on the permuted sequential MNIST, with or without black pixels added at the end of the sequences, and for different ratios of BMRU and LRU neurons in each recurrent layer. $X - Y$ must be read as X BMRU neurons for Y LRU neurons. All models have two recurrent layers with a size of 256.

Appendix F. Making the BMRU parameter α input-dependent



(a) BMRU alone



(b) Hybrid model

Fig. F.15. Impact of an input-dependent α on the performance of BMRU. Accuracies obtained by BMRU and hybrid models on the permuted sequential MNIST, with or without black pixels added at the end of the sequences, by considering an input-dependent or input-independent α in the equations of BMRU. All models have two recurrent layers with a size of 256.

The parameter α is introduced in Eq. (8) as the value of the stable state toward which the hysteretic system emulated by BMRU converges. This value is independent on the inputs, which is why α is a learned parameter in BMRU and not a function of the current input. However, from a ML perspective, making it dependent of the current input could improve BMRU's ability to encode information from the specific timestep. This appendix evaluates how adding such dependency can improve or impede performance of BMRU and hybrid models on the permuted sequential MNIST, with and without black pixels. Mathematically, the equations of the modified BMRU are:

$$\hat{h}_t = W_x x_t + b_x,$$
$$\beta_t = |W_\beta x_t + b_\beta|,$$
$$z_t = H(|\hat{h}_t| - \beta_t),$$
$$\alpha_t = W_\alpha x_t + b_\alpha,$$
$$h_t = z_t \odot S(\hat{h}_t) \odot \alpha_t + (1 - z_t) \odot h_{t-1},$$

Fig. F.15 shows the mean accuracies obtained for both BMRU and hybrid models and for an input-dependent and independent α. Adding such dependency seems to deteriorate BMRU performance when it is used alone, while performance is unaffected for the hybrid model. Therefore, it seems that the quantization imposed by the hysteretic behavior does not impede performance in this task. However, this could not be true in tasks where more information is provided at every timestep, in which case encoding input-dependent values might be more helpful.

References

Bai, S., Kolter, J. Z., & Koltun, V. (2019). Deep equilibrium models. In *Advances in neural information processing systems*. Curran Associates, Inc. (*vol. 32*).

Bakija, A., De Geeter, F., Brandoit, J., Sacré, P., & Drion, G. (2026). On the Importance of Multistability for Horizon Generalization in Reinforcement Learning. https://doi.org/10.48550/arxiv.2605.12206

Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). xLSTM: extended long short-Term memory. *Advances in Neural Information Processing Systems*, *37*, 107547–107603. https://doi.org/10.52202/079017-3417

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. https://doi.org/10.48550/arXiv.1308.3432

Blelloch, G. E. (1990). Prefix Sums and Their Applications. Technical Report School of Computer Science, Carnegie Mellon University Pittsburgh, PA, USA.

Boyd, S., & Chua, L. (1985). Fading memory and the problem of approximating nonlinear operators with Volterra series. *IEEE Transactions on Circuits and Systems*, *32*(11), 1150–1161. https://doi.org/10.1109/TCS.1985.1085649

Brandoit, J., Fyon, A., Ernst, D., & Drion, G. (2026). Improving the Performance and Learning Stability of Parallelizable RNNs Designed for Ultra-Low Power Applications. https://doi.org/10.48550/arXiv.2605.11855

Cheng, C.-Y., Lin, K.-H., & Shih, C.-W. (2006). Multistability in recurrent neural networks. *SIAM Journal on Applied Mathematics*, *66*(4), 1301–1320. https://doi.org/10.1137/050632440

Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1724–1734). Doha, Qatar: Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1179

Danieli, F., Rodriguez, P., Sarabia, M., Suau, X., & Zappella, L. (2025). ParaRNN: unlocking parallel training of nonlinear RNNs for large language models. In *The fourteenth international conference on learning representations*.

Edwards, R. (2000). Analysis of continuous-time switching networks. *Physica D: Nonlinear Phenomena*, *146*(1–4), 165–199. https://doi.org/10.1016/S0167-2789(00)00130-5

Eshraghian, J. K., Ward, M., Neftci, E., Wang, X., Lenz, G., Dwivedi, G., Bennamoun, M., Jeong, D. S., & Lu, W. D. (2023). Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, *111*(9), 1016–1054. https://doi.org/10.1109/JPROC.2023.3308088

Feng, L., Tung, F., Ahmed, M. O., Bengio, Y., & Hajimirsadeghi, H. (2024). Were RNNs All We Needed? Available at https://arxiv.org/abs/2410.01201.

Fyon, A., Brandoit, J., Mendolia, L., Ernst, D., Redouté, J.-M., & Drion, G. (2026). Hardware-Software Co-Design of Scalable, Energy-Efficient Analog Recurrent Computations. https://doi.org/10.48550/arxiv.2605.15216

Gonzalez, X., Warrington, A., Smith, J. T., & Linderman, S. W. (2024). Towards scalable and stable parallelization of nonlinear RNNs. *Advances in Neural Information Processing Systems*, *37*, 5817–5849. https://doi.org/10.52202/079017-0189

Goodman, R. M., & Zeng, Z. (1994). A learning algorithm for multi-layer perceptrons with hard-limiting threshold units. In *Proceedings of IEEE workshop on neural networks for signal processing* (pp. 219–228). https://doi.org/10.1109/NNSP.1994.366045

Gu, A., & Dao, T. (2024). Mamba: linear-time sequence modeling with selective state spaces. In *First conference on language modeling*.

Gu, A., Goel, K., & Re, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces. In *International Conference on Learning Representations*.

Gu, A., Johnson, I., Goel, K., Saab, K. K., Dao, T., Rudra, A., & Re, C. (2021). Combining recurrent, convolutional, and continuous-time models with linear state-space layers. In *Advances in Neural Information Processing Systems*, *34*, (pp. 572–585).

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Jiang, H., Qin, F., Cao, J., Peng, Y., & Shao, Y. (2021). Recurrent neural network from adder's perspective: Carry-lookahead RNN. *Neural Networks*, *144*, 297–306. https://doi.org/10.1016/j.neunet.2021.08.032

Krishnamurthy, K., Can, T., & Schwab, D. J. (2022). Theory of gating in recurrent neural networks. *Physical Review X*, *12*(1), 011011. https://doi.org/10.1103/PhysRevX.12.011011

Lambrechts, G., De Geeter, F., Vecoven, N., Ernst, D., & Drion, G. (2023). Warming up recurrent neural networks to maximise reachable multistability greatly improves learning. *Neural Networks*, *166*, 645–669. https://doi.org/10.1016/j.neunet.2023.07.023

Le, Q. V., Jaitly, N., & Hinton, G. E. (2015). A Simple Way to Initialize Recurrent Networks of Rectified Linear Units. ArXiv.

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324. https://doi.org/10.1109/5.726791

Lee, J. H., Delbruck, T., & Pfeiffer, M. (2016). Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, *10*.

Lim, Y. H., Zhu, Q., Selfridge, J., & Kasim, M. F. (2023). Parallelizing non-linear sequential models over the sequence length. In *The twelfth international conference on learning representations* (pp. 55334–55360).

Martin, E., & Cundy, C. (2018). Parallelizing linear recurrent neural nets over sequence length. In *International conference on learning representations*.

Merrill, W., Petty, J., & Sabharwal, A. (2024). The illusion of state in state-space models. In *Proceedings of the 41st international conference on machine learning* (pp. 35492–35506). Vienna, Austria: JMLR.org (*vol. 235*). ICML'24.

Neftci, E., Mostafa, H., & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, *36*(6), 51–63. https://doi.org/10.1109/MSP.2019.2931595

Nieves, N. P., & Goodman, D. F. M. (2021). Sparse spiking gradient descent. In *Neural information processing systems*.

Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., & De, S. (2023). Resurrecting recurrent neural networks for long sequences. In *Proceedings of the 40th international conference on machine learning* (pp. 26670–26698). PMLR.

Qin, Z., Yang, S., & Zhong, Y. (2023). Hierarchically gated recurrent neural network for sequence modeling. *Advances in Neural Information Processing Systems*, *36*, 33202–33221.

Shan, D., Luo, Y., Zhang, X., & Zhang, C. (2023). DRRNets: Dynamic recurrent routing via low-rank regularization in recurrent neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *34*(4), 2057–2067. https://doi.org/10.1109/TNNLS.2021.3105818

Smith, J. T. H., Warrington, A., & Linderman, S. (2022). Simplified state space layers for sequence modeling. In *The eleventh international conference on learning representations*.

Sun, P., Jiang, W., Devos, P., & Botteldooren, D. (2025). Parallel delayed memory units for enhanced temporal modeling in biomedical and bioacoustic signal analysis. *IEEE Transactions on Audio, Speech and Language Processing*, *33*, 4797–4807. https://doi.org/10.1109/TASLPRO.2025.3633044

Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., & Metzler, D. (2020). Long range arena : a benchmark for efficient transformers. In *International conference on learning representations*.

Tournoy, M., & Doiron, B. (2022). A Step Towards Uncovering The Structure of Multistable Neural Networks. Available at https://arxiv.org/abs/2210.03241.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems*. Curran Associates, Inc. (*vol. 30*).

Vecoven, N., Ernst, D., & Drion, G. (2021). A bio-inspired bistable recurrent cell allows for long-lasting memory. *PloS One*, *16*(6), e0252676. https://doi.org/10.1371/journal.pone.0252676

Yin, B., Wang, S., Tan, H., Bohte, S., Corradi, F., & Li, G. (2026). Efficient Sparse Selective-Update RNNs for Long-Range Sequence Modeling. https://doi.org/10.48550/arXiv.2603.02226

Zeng, Z., Goodman, R. M., & Smyth, P. (1993). Learning finite state machines with self-Clustering recurrent networks. *Neural Computation*, *5*(6), 976–990. https://doi.org/10.1162/neco.1993.5.6.976

Zhang, Z., Shen, L., Zhang, S., Ye, D., Luo, Y., Shi, M., Shan, D., Du, B., & Tao, D. (2026). Aligning few-step diffusion models with dense reward difference learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *48*(7), 7375–7386. https://doi.org/10.1109/TPAMI.2026.3665753

Zini, J. E., Rizk, Y., & Awad, M. (2021). An optimized parallel implementation of non-iteratively trained recurrent neural networks. *Journal of Artificial Intelligence and Soft Computing Research*, *11*(1), 33–50. https://doi.org/10.2478/jaiscr-2021-0003